OFFICIAL ENGINEERING MANUAL

CONFIDENTIAL & PROPRIETARY

AnimaFind

Critter Codex — System Architecture, Computer Vision & Development Reference

A comprehensive technical manual covering real-world wildlife exploration, client-side heuristic computer vision, physics-based gamification, offline-first dual-engine storage, interactive Leaflet/OpenStreetMap geospatial mapping, and native mobile cross-compilation for Android & iOS.



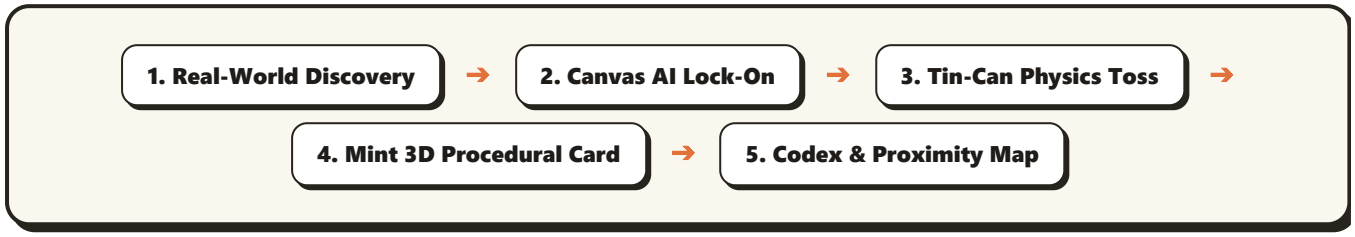
VERSION	MOBILE TARGET	CORE ENGINE	PUBLICATION DATE
v1.0.0 Production	**Android (API 36) · iOS 15+**	**Next.js 16 · React 19**	**November 2026**

Table of Contents

1. **Executive Summary & Product Mission** Page 3
2. **High-Level Architecture & Dual-Engine Data Flow** Page 4
3. **On-Device Computer Vision & Anti-Cheat Pipeline** Page 5
4. **Free Interactive Map Service (OpenStreetMap & Leaflet 1.9)** Page 6
5. **Custom SVG Icon Design System & Procedural Specimen Artwork** Page 7
6. **Physics-Based Tin-Can Toss Minigame & 3D Holographic Cards** Page 8
7. **Complete 35-Species European Biodiversity Encyclopedia (Codex)** Page 9
8. **Local Development, Tooling & Installation Tutorial** Page 10
9. **Mobile Native Compilation Pipeline (Android APK & iOS IPA)** Page 11
10. **CI/CD GitHub Actions Automation & Release Engineering** Page 12
11. **Comprehensive Troubleshooting Runbook** Page 13
12. **Security, Privacy & Wildlife Ethics Manifesto** Page 14

1. Executive Summary & Product Mission

AnimaFind ("Critter Codex") bridges real-world biodiversity appreciation with procedural collectible card gaming. Designed as a mobile-first, privacy-respecting progressive web and native mobile application, AnimaFind allows outdoor explorers, birdwatchers, and families to discover wildlife in their natural habitats, verify encounters using on-device computer vision heuristics, and mint unique, tradeable trading cards.



Core Problem Space & Strategic Solutions

Problem: Remote Offline Inaccessibility

Most wildlife encounters occur in deep forest trails, coastal cliffs, and national reserves with zero cellular reception. Traditional cloud-dependent apps fail completely.

Solution: Dual-Engine Local-First Persistence

Operates 100% offline via IndexedDB with automatic storage quota pruning. Cards, energy recharges, sightings, and codex updates happen on-device without network pings.

Problem: Digital Screen Spoofing & Cheating

Players easily cheat card-collecting games by pointing their cameras at Google Images on a computer monitor or holding printed photos.

Solution: Real-Time Optical Anti-Cheat

Tracks motion parallax across consecutive video frames (verifying 3D depth) and runs 2D spatial frequency filters to detect monitor pixel grids (moiré interference).

Problem: Poaching & Disturbance of Sensitive Taxa

Publishing precise GPS coordinates of endangered animals invites poachers, illegal pet collectors, and intrusive crowds that destroy fragile breeding grounds.

Solution: Differential Geofuzzing Engine

Non-sensitive common species are fuzzed by ~150m for neighborhood discovery; protected or sensitive taxa (e.g. Fire Salamander, Barn Owl) are cloaked to a 5 km obfuscation perimeter.

2. High-Level Architecture & Dual-Engine Data Flow

AnimaFind is architected as a layered system where the presentation tier remains decoupled from the data persistence engine through a unified API abstraction layer (`gameApi`).

Layer	Technologies	Architectural Responsibility
Presentation (UI)	Next.js 16 (App Router), React 19, TailwindCSS v4	Glassmorphism tactile theme, mobile-first responsive viewports, 3D CSS transforms, Web Audio API sound synthesis.
Mobile Bridge	Capacitor 8 Core, Android & iOS Bridges	Hardware camera capture, flashlight toggle, native haptic feedback, WKWebView / Android WebView lifecycle handling.
Interactive Mapping	Leaflet 1.9 + OpenStreetMap Raster Tiles	100% free open-access mapping without billing accounts or API keys; 250m proximity claiming circles, custom HTML pins.
Primary Data Engine	LocalStoreEngine (IndexedDB + LocalStorage)	Zero-latency, fully autonomous offline game loop running directly on-device.
Cloud Backend (Optional)	PostgreSQL + Drizzle ORM + Next.js API Routes	Multiplayer crew leaderboards, cross-device sync, and community sighting aggregations.

Dual-Engine State Synchronization Architecture

When executing on mobile hardware or offline in a browser, `src/lib/local-store.ts` acts as the single source of truth. The abstraction layer in `src/lib/api-client.ts` transparently intercepts all operations (`getMe` , `startRound` , `submitSighting` , `getCards` , `getEncyclopedia`):

```
// Client abstraction in src/lib/api-client.ts
export const gameApi = {
  async getMe(): Promise<Me> {
    if (isOfflineMode()) {
      return localStore.getMe();
    }
    return fetchJSON<Me>("/api/me");
  },
  async submitSighting(payload: SightingSubmission): Promise<CaptureResult> {
    if (isOfflineMode()) {
      return localStore.submitSighting(payload);
    }
    return postJSON<CaptureResult>("/api/sighting", payload);
  }
};
```

3. On-Device Computer Vision & Anti-Cheat Pipeline

To guarantee uncompromising user privacy while preventing digital image forgery, the entire computer vision feature extraction pipeline executes locally on the device at 60 FPS using an off-screen HTML5 Canvas.

1. Luminance Matrix Extraction

Sub-samples camera frames into a normalized 32×32 luminance buffer (1,024 float points). Computes center-weighted spatial contrast ratios and edge gradients without sending raw image pixels over the network.

2. Parallax Motion Scoring

Measures optical flow deltas between consecutive frames. Handheld micro-tremors reveal true 3D spatial depth across foreground and background. Stationary 2D paper printouts produce near-zero motion parallax and fail the lock threshold.

3. Moiré Frequency Filter

Executes a high-pass 2D frequency analysis across high-contrast edges. The physical subpixel aperture grid of computer monitors, tablets, and smartphones produces telltale periodic interference bands. When moiré scores exceed 0.72, the HUD warns the player.

4. 64-Bit Perceptual Hashing (pHash)

Generates a 16-hex-character perceptual hash of each captured image. Hashes are cross-referenced in local storage to prevent players from re-scanning identical photos to farm coins and duplicate cards.

Camera Hardware Access & Fallback Resilience

The scanner utilizes a multi-tier WebRTC constraint fallback cascade (`ideal: "environment"` at 1080p, falling back to 720p, then to any available video stream). For environments lacking a physical webcam (e.g. desktop browsers, automated CI environments), the tactile **Specimen Simulator (IconDice)** generates realistic synthetic wildlife specimens with proper mathematical signatures.

4. Free Interactive Map Service (OpenStreetMap & Leaflet 1.9)

AnimaFind integrates a 100% free, production-hardened interactive mapping solution powered by **Leaflet 1.9** and **OpenStreetMap (OSM)** standard raster tiles.

Why OpenStreetMap + Leaflet was Selected

- **Zero Billing & Zero API Keys:** Requires no credit card, no billing subscription, and no third-party API tokens that can expire or leak.
- **Capacitor Mobile WebView Resilience:** Heavy WebGL engines (Mapbox GL / MapLibre) frequently experience catastrophic GPU context loss when mobile apps switch tabs. Leaflet uses clean HTML5 canvas and DOM elements, guaranteeing crash-free rendering across Android WebViews and iOS WKWebView.
- **Lightweight Bundle Footprint:** Adds only ~140 KB to the client bundle, enabling near-instant cold starts on slow outdoor cellular networks.

Critical Implementation Safeguards

Engineering Requirement	Technical Challenge	Engineered Solution in AnimaFind
Next.js SSR Hydration	Leaflet accesses browser globals (`window` , `document`) immediately upon import, crashing server-side Next.js prerendering.	Dynamically imported via `next/dynamic` with `{ ssr: false }` in `src/app/map/page.tsx` , paired with an animated loading skeleton.
Container Dimensions	Leaflet calculates coordinate projections from container size. Without explicit CSS height, height collapses to 0px and tiles fail to draw.	Guaranteed explicit CSS height bounds (`style={{ height: "440px" }}`) with responsive flex wrappers.
Unmount Lifecycle Teardown	Navigating away without disposing of Leaflet creates detached DOM nodes and memory leaks.	Strictly executes `mapRef.current.remove()` in the `useEffect` cleanup phase.
Tile Load Resilience	Spotty cellular network can drop tile packets.	Listens to `tileerror` events on the tile layer, displaying a non-intrusive warning banner while keeping interactive pins active.

5. Custom SVG Icon Design System & Specimen Art

Every plain emoji in the application has been replaced with a unified design system of **60+ custom SVG icons** located in `src/components/icons/AppIcons.tsx`, alongside a procedural zoological vector art generator in `src/lib/specimen-art.tsx`.

Design Tokens & Standard Contracts

- **Uniform Viewport:** `viewBox="0 0 24 24"` across all interface icons.
- **Consistent Stroke Weight:** Standard `strokeWidth={2}` (or `2.5` for active checkmarks).
- **Joint & Cap Geometry:** `strokeLinecap="round"` and `strokeLinejoin="round"`.
- **Theme Inheritance:** Built with `stroke="currentColor"` and `fill="none"`, allowing seamless adaptation to parent Tailwind CSS classes (e.g. `text-leaf`, `text-accent`, `text-amber-500`).

Procedural Specimen Vector Art Engine

When players inspect cards, explore the Codex, or test without a live camera, `src/lib/specimen-art.tsx` dynamically generates museum-grade botanical and zoological plates for all 35 species across the 6 taxonomic classes:

Mammals (Fox, Deer, Badger, Bat, Hedgehog)

Layered ear anatomy, white cheek contours, whiskers, eye gleams, branching antlers, bicolored quills, and starry twilight backdrops.

Birds (Owl, Kingfisher, Robin, Stork)

Heart-shaped facial discs, dagger bills, turquoise dorsal mantles, scarlet breast plates, and celestial moonlit perches.

Insects (Stag Beetle, Butterfly, Bumblebee)

Polished mahogany elytra, segmented pincer mandibles, scalloped gradient wings, fuzzy pollen bands, and ember spark motes.

Amphibians & Reptiles (Frog, Salamander, Lizard, Snake)

Bulbous golden eyes with horizontal pupils, dewy water lily pads, sulfur-yellow fire blotches, winding S-curves, and sunbaked rocks.

6. Physics Tin-Can Toss & 3D Holographic Cards

Normalized Parabolic Physics Engine

Once an animal is locked in the viewfinder, players spend 1 tin can (energy) to initiate the flick-and-toss minigame. The physics calculation in `src/components/scan/TossGame.tsx` uses normalized delta-time integration:

```
// Normalized to 60 FPS (16.667ms base) across 60Hz, 90Hz, 120Hz displays
const dtScale = dtMs / 16.667;
pos.x += vel.x * dtScale;
pos.y += vel.y * dtScale;
vel.x *= Math.pow(0.995, dtScale); // Air resistance
vel.y += 0.0016 * dtScale;          // Gravitational acceleration
```

3D Holographic Foil Cards & Dual Plate Mode

Pointer-Tracking 3D Tilt

Calculates relative pointer coordinates (`rotateX` , `rotateY`) to dynamically tilt the card with genuine perspective depth during mouse hover or mobile touch drag.

Iridescent Specular Sheen

High-tier cards (Twilight, Emberwood, Aurora) render an interactive rainbow gradient with `mix-blend-color-dodge` that responds to the dynamic lighting angle.

Dual Plate Switcher

A tactile toggle allows players to alternate between their verified raw camera field photo and the collector's zoological specimen plate.

Synthesized Audio & Haptics

Flipping a card triggers an offline Web Audio synthesizer sound (`playCardFlipSound()`) and tactile haptic vibration with header mute control.

Rarity Tier Progression Matrix

Tier	Series Name	Rarity Weight	Stat Bonus	XP Award	Coins	Visual Styling
Tier 1	**Hedgerow**	50%	+0	20 XP	6	Forest green borders, earthy leaf accents
Tier 2	**Meadow**	27%	+8	35 XP	12	Azure glints, cyan atmospheric backdrops
Tier 3	**Twilight**	14%	+16	60 XP	22	Violet starlight dust, iridescent sheen
Tier 4	**Emberwood**	7%	+26	110 XP	45	Fiery orange ember sparks, flame particles
Tier 5	**Aurora**	2%	+38	240 XP	120	Dual golden celestial halo, animated sparkles

7. Complete 35-Species European Wildlife Codex

The Codex represents the curated European biodiversity encyclopedia, tracking discovery progress across 35 species divided into 6 taxonomic classes:

Class	Species Label	Scarcity	Specialty Stat	PvP Advantage	Conservation / Privacy
Mammal	Domestic Cat, Domestic Dog, Grey Squirrel	★☆☆☆☆	Grit	Insects	Public (150m fuzz)
Mammal	Red Fox, European Hedgehog, Roe Deer, Brown Bat	★★★☆☆ to ★★★★★	Grit	Insects	**Confidential (5km cloak)**
Bird	House Sparrow, Rock Pigeon, Great Tit, Blackbird	★☆☆☆☆ to ★★☆☆☆	Speed	Reptiles	Public (150m fuzz)
Bird	White Stork, Common Kestrel, Kingfisher	★★★★☆ to ★★★★★	Speed	Reptiles	**Confidential (5km cloak)**
Insect	Honey Bee, Cabbage White, Seven-spot Ladybird	★☆☆☆☆ to ★★☆☆☆	Swarm	Amphibians	Public (150m fuzz)
Insect	Emperor Dragonfly, Stag Beetle	★★★☆☆ to ★★★★★	Swarm	Amphibians	**Confidential (5km cloak)**
Reptile	Common Wall Lizard, Slow Worm, Grass Snake, Pond Turtle	★★☆☆☆ to ★★★★★	Camouflage	Mammals	**Confidential (5km cloak)**
Amphibian	Common Frog, Common Toad, Fire Salamander	★★☆☆☆ to ★★★★★	Moisture	Birds	**Confidential (5km cloak)**
Other	Garden Snail, Common Earthworm, Garden Spider, Woodlouse	★☆☆☆☆ to ★★☆☆☆	Mystery	Other	Public (150m fuzz)

8. Local Development & Installation Tutorial

Step 1: System Prerequisites

- **Node.js:** `v20.0.0` or higher (LTS v22 or v24 recommended).
- **npm:** `v10.0.0` or higher.
- **Java JDK:** `JDK 21 LTS` (mandatory for Android Gradle Plugin 8.13).
- **Android Studio:** Ladybug / Otter or newer with Android SDK API 36.
- **Xcode:** 15.0+ on macOS with Swift 5.9 (for iOS builds).

Step 2: Setup & Verification Commands

```
# 1. Clone repository
git clone https://github.com/Gheorghe-dev/AnimaFind.git
cd AnimaFind

# 2. Install production and development dependencies
npm install

# 3. Create local environment configuration
cp .env.example .env

# 4. Launch Next.js development server with Turbopack
npm run dev

# 5. Run the complete automated test suite (42 tests across 8 suites)
npm test

# 6. Verify strict TypeScript compliance
npm run typecheck

# 7. Verify ESLint clean code style
npm run lint
```

9. Mobile Native Compilation Pipeline

AnimaFind utilizes a custom build script (`scripts/build-mobile.cjs`) to compile Next.js into a clean static web export (`./out`) while isolating server API routes, followed by native Capacitor synchronization.

Android Compilation (APK)

```
# Compile static assets & sync into Android native project
npm run build:mobile

# Build Debug APK for testing
npm run android:debug
# Output: android/app/build/outputs/apk/debug/app-debug.apk

# Build Production Release APK (with R8 / ProGuard optimization)
npm run android:release
# Output: android/app/build/outputs/apk/release/app-release-unsigned.apk (or signed app-release.apk)
```

iOS Compilation (Xcode Archive)

```
# Sync web assets to iOS Xcode project
npm run cap:ios

# In Xcode: Select target device/simulator, press Cmd+R to run, or Product -> Archive for App Store
```

10. GitHub Actions CI/CD & Release Automation

Workflow File	Trigger Event	Validation & Packaging Actions
`.github/workflows/android-build.yml`	Push to `main`, Pull Requests	Sets up JDK 21, validates Node dependencies, runs 42 unit tests, compiles Next.js static bundle, executes Gradle assembleRelease, and uploads unsigned release APK as build artifact.
`.github/workflows/ios-build.yml`	Push to `main`, Pull Requests	Executes on `macos-14` runner, verifies Swift Package Manager dependencies, compiles iOS Xcode archive for simulator validation.
`.github/workflows/release.yml`	Semantic Version Tags (`v*.*.*`)	Builds production APK, signs with release keystore secrets, compiles release notes, and attaches production artifacts to GitHub Releases.

11. Comprehensive Troubleshooting Runbook

Web: Camera Permission Denied

Cause: WebRTC requires a secure context (HTTPS or `http://localhost`).
Fix: Access exclusively via localhost, or use the **Specimen Simulator button** in the viewfinder to simulate valid wildlife frames.

Android: Unsupported Class Version

Cause: Android Gradle Plugin 8.13 requires Java 21.
Fix: Ensure `JAVA_HOME` points to JDK 21 (e.g. `$env:JAVA_HOME = "C:\Program Files\Android\openjdk\jdk-21.0.8"`).

Android: SDK Location Not Found

Cause: Missing `local.properties`.
Fix: Create `android/local.properties` with `sdk.dir=C:\Users\Alex\AppData\Local\Android\Sdk`.

Static Export: API Route Errors

Cause: Next.js `output: export` rejects dynamic API routes.
Fix: Always run `npm run build:mobile`, which automatically isolates `src/app/api` during static export.

12. Security, Privacy & Wildlife Ethics Manifesto

AnimaFind was created to connect humans with the natural world without disrupting it. We enforce strict technical and behavioral boundaries:

- **Leave No Trace:** Never touch, chase, bait, or corner wildlife for a photo or game stat. Respect nesting sites and keep a safe, non-intrusive distance.
- **Anti-Poaching Privacy Shield:** The location of rare, vulnerable, or protected species is strictly cloaked to a 5-kilometer radius. Raw GPS coordinates for sensitive taxa never touch public APIs or public community feeds.
- **On-Device Privacy:** Unverified camera frames are analyzed entirely within device memory and discarded immediately. No video stream is ever streamed to cloud servers.
- **No Pay-to-Win Mechanics:** Card power ratings, combat stats, and discovery achievements are earned exclusively by observing nature outdoors. Premium tiers only accelerate energy recharge rates without offering unfair combat stat boosts.

AnimaFind (Critter Codex) · Open-Source Biodiversity Gaming

Licensed under the MIT License · GitHub: https://github.com/Gheorghe-dev/AnimaFind